Week 1 Structured Notes: Recursion Foundations

1. Week 1 Main Goal

The goal of Week 1 is to understand the basic idea of recursion. By the end of this week, you should be able to explain what recursion is, why recursion must stop, and how recursive function calls behave when they go deeper and then return back.

Week 1 focuses on these main concepts:

- Normal function calls
- Recursive functions
- Base condition or base case
- Recursive case
- Calling phase
- Returning phase
- Tracing recursive functions
- Predicting program output before running code

---

2. What Is Recursion?

Recursion means a function calls itself.

In simple words, recursion is when a function repeats its own work by calling itself again with a smaller or simpler input.

Example idea:

```
printDesc(3);
```

This can lead to:

```
printDesc(3)
printDesc(2)
printDesc(1)
printDesc(0)
```

Each call works with a smaller value of `n`.

The function continues until it reaches a stopping point.

That stopping point is called the base case.

---

3. Why Learn Normal Function Calls First?

Before learning recursion, you must understand how normal function calls work.

A program usually starts from the `main()` function. When `main()` calls another function, the program temporarily leaves `main()` and goes to that function. After that function finishes, the program returns to the exact place where the function was called.

Example:

```c
#include <stdio.h>

void greet() {
    printf("Hello\n");
}

int main() {
    greet();
    printf("Done\n");
    return 0;
}
```

Output:

```
Hello
Done
```

Explanation:

1. The program starts in `main()`.
2. `main()` calls `greet()`.
3. The program enters the `greet()` function.
4. `greet()` prints `Hello`.
5. `greet()` finishes and returns to `main()`.
6. `main()` continues and prints `Done`.

Simple example:

Think of `main()` as your home. When you call another function, you visit another room. After finishing work in that room, you come back home and continue from where you stopped.

---

4. Recursive Function

A recursive function is a function that calls itself.

Example:

```
void example() {
    example();
}
```

This is recursion because `example()` calls itself.

But this code is dangerous because it has no stopping condition.

It will continue like this:

```
example calls example
example calls example
example calls example
example calls example
...
```

This will eventually crash the program because the function keeps calling itself without stopping.

So, every recursive function must have a base case.

---

5. Two Main Parts of Recursion

Every correct recursive function has two main parts.

5.1 Base Case

The base case is the condition that stops recursion.

It tells the function:

```
Stop calling yourself now.
```

Without a base case, recursion can continue forever until the program crashes.

5.2 Recursive Case

The recursive case is the part where the function calls itself again.

It usually calls itself with a smaller or simpler value.

Example:

```
void countDown(int n) {
    if (n > 0) {
        printf("%d ", n);
        countDown(n - 1);
    }
}
```

In this example:

```
if (n > 0)
```

is the condition that controls recursion.

When `n` becomes `0`, the condition becomes false, and the function stops calling itself.

---

6. Base Condition

The base condition is the safety rule of recursion.

It prevents the function from calling itself forever.

Example:

```
void printDesc(int n) {
    if (n > 0) {
        printf("%d ", n);
        printDesc(n - 1);
    }
}
```

Function call:

```
printDesc(3);
```

Trace:

```
printDesc(3) prints 3 and calls printDesc(2)
printDesc(2) prints 2 and calls printDesc(1)
printDesc(1) prints 1 and calls printDesc(0)
printDesc(0) stops because n > 0 is false
```

Output:

```
3 2 1
```

Why did it stop?

It stopped because when `n = 0`, this condition becomes false:

```
if (n > 0)
```

So the function does not call itself again.

---

7. Calling Phase

The calling phase is the part of recursion where function calls are going deeper.

In this phase, the function keeps calling itself again and again until it reaches the base case.

Look at this code:

```
#include <stdio.h>

void printDesc(int n) {
    if (n > 0) {
        printf("%d ", n);
        printDesc(n - 1);
    }
}

int main() {
    printDesc(3);
```

```
    return 0;
}
```

Important line order:

```
printf("%d ", n);
printDesc(n - 1);
```

Here, printing happens before the recursive call.

So the work happens during the calling phase.

Trace:

```
printDesc(3): print 3
printDesc(2): print 2
printDesc(1): print 1
printDesc(0): stop
```

Output:

```
3 2 1
```

Simple rule:

If the work happens before the recursive call, the work happens during the calling phase.

Example:

```
printf("%d ", n);     // work first
printDesc(n - 1);     // recursive call second
```

Because the print statement comes first, the output appears while the function is going deeper.

---

8. Returning Phase

The returning phase is the part where recursive calls finish and return back one by one.

Look at this code:

```c
#include <stdio.h>

void printAsc(int n) {
    if (n > 0) {
        printAsc(n - 1);
        printf("%d ", n);
    }
}

int main() {
    printAsc(3);
    return 0;
}
```

Important line order:

```c
printAsc(n - 1);
printf("%d ", n);
```

Here, the recursive call happens before printing.

So the function goes deeper first without printing. After reaching the base case, it returns back and then prints values.

Trace while going deeper:

```
printAsc(3) calls printAsc(2)
printAsc(2) calls printAsc(1)
printAsc(1) calls printAsc(0)
printAsc(0) stops
```

Trace while returning:

```
printAsc(1) prints 1
printAsc(2) prints 2
printAsc(3) prints 3
```

Output:

```
1 2 3
```

Simple rule:

If the work happens after the recursive call, the work happens during the returning phase.

Example:

```
printAsc(n - 1);      // recursive call first
printf("%d ", n);     // work second
```

Because the print statement comes after the recursive call, the output appears while the function is returning.

---

9. Difference Between Calling Phase and Returning Phase

9.1 Work Before Recursive Call

```
void printDesc(int n) {
    if (n > 0) {
        printf("%d ", n);
        printDesc(n - 1);
    }
}
```

Output for `printDesc(3)`:

```
3 2 1
```

Reason:

The print statement runs before the recursive call. So the values are printed while recursion is going down.

9.2 Work After Recursive Call

```
void printAsc(int n) {
    if (n > 0) {
        printAsc(n - 1);
        printf("%d ", n);
    }
}
```

Output for `printAsc(3)`:

```
1 2 3
```

Reason:

The print statement runs after the recursive call. So the values are printed while recursion is coming back.

---

10. Simple Mental Model

Think of recursion like walking down stairs.

printDesc(3)

For `printDesc(3)`, the function speaks while going down:

```
3
2
1
```

So the output is:

```
3 2 1
```

printAsc(3)

For `printAsc(3)`, the function walks down silently first. Then it speaks while coming back up:

```
1
2
3
```

So the output is:

```
1 2 3
```

This is the main difference between the calling phase and the returning phase.

---

11. Combined Example: Before and After Recursive Call

This example has work both before and after the recursive call.

```c
#include <stdio.h>

void test(int n) {
    if (n > 0) {
        printf("Before %d\n", n);
        test(n - 1);
        printf("After %d\n", n);
    }
}

int main() {
    test(3);
    return 0;
}
```

Output:

```
Before 3
Before 2
Before 1
After 1
After 2
After 3
```

Explanation:

The first print statement happens before the recursive call:

```c
printf("Before %d\n", n);
```

So it runs during the calling phase.

The second print statement happens after the recursive call:

```c
printf("After %d\n", n);
```

So it runs during the returning phase.

Trace:

```
test(3): prints Before 3, then calls test(2)
test(2): prints Before 2, then calls test(1)
test(1): prints Before 1, then calls test(0)
test(0): stops

test(1): prints After 1
test(2): prints After 2
test(3): prints After 3
```

---

12. How to Trace a Recursive Function

When tracing recursion, do not run the code first.

Follow this method:

1. Write the function call.
2. Check the base condition.
3. Identify what happens before the recursive call.
4. Write the next recursive call.
5. Continue until the base case is reached.
6. Then trace what happens while returning.
7. Write the final output.

Example:

```
printDesc(3);
```

Trace:

```
printDesc(3) -> print 3 -> call printDesc(2)
printDesc(2) -> print 2 -> call printDesc(1)
printDesc(1) -> print 1 -> call printDesc(0)
printDesc(0) -> stop
```

Final output:

```
3 2 1
```

---

13. Important Terms

Recursion

A method where a function calls itself.

Recursive Function

A function that calls itself.

Base Case

The condition that stops recursion.

Recursive Case

The part where the function calls itself again.

Calling Phase

The stage where recursive calls are going deeper.

Returning Phase

The stage where recursive calls finish and return back.

Dry Run

Manually checking the program step by step before running it.

---

14. Common Beginner Mistakes

Mistake 1: Forgetting the Base Case

Wrong example:

```
void example() {
    example();
}
```

Problem:

This function never stops.

Mistake 2: Not Reducing the Input

Wrong example:

```
void countDown(int n) {
    if (n > 0) {
        printf("%d ", n);
        countDown(n);
    }
}
```

Problem:

`n` never changes, so the function keeps calling itself with the same value.

Correct version:

```
void countDown(int n) {
    if (n > 0) {
        printf("%d ", n);
        countDown(n - 1);
    }
}
```

Mistake 3: Thinking Output Always Happens Immediately

In recursion, output depends on where the print statement is placed.

If printing is before the recursive call, output happens while going down.

If printing is after the recursive call, output happens while returning.

---

15. Week 1 Practice Programs

Program 1: Print Numbers in Descending Order

```
#include <stdio.h>

void printDesc(int n) {
    if (n > 0) {
        printf("%d ", n);
        printDesc(n - 1);
```

```
    }
}

int main() {
    printDesc(5);
    return 0;
}
```

Expected output:

```
5 4 3 2 1
```

Reason:

The print statement is before the recursive call.

Program 2: Print Numbers in Ascending Order

```
#include <stdio.h>

void printAsc(int n) {
    if (n > 0) {
        printAsc(n - 1);
        printf("%d ", n);
    }
}

int main() {
    printAsc(5);
    return 0;
}
```

Expected output:

```
1 2 3 4 5
```

Reason:

The print statement is after the recursive call.

---

16. Week 1 Summary

Recursion is when a function calls itself. A recursive function must have a base case to stop it. Without a base case, the function can continue forever and crash the program.

A recursive function usually has two parts: the base case and the recursive case. The base case stops the recursion. The recursive case calls the same function again with a smaller input.

The calling phase happens when recursive calls are going deeper. If the work is written before the recursive call, it happens during the calling phase.

The returning phase happens when recursive calls finish and come back. If the work is written after the recursive call, it happens during the returning phase.

To understand recursion properly, you should trace the code manually before running it. Recursion becomes clear when you draw or write the function calls step by step.

---

17. Week 1 Mastery Questions

Answer these before moving to Week 2:

1. What is recursion?
2. What is a recursive function?
3. What is a base case?
4. Why does recursion need a base case?
5. What is the recursive case?
6. What happens if a recursive function has no stopping condition?
7. What is the calling phase?
8. What is the returning phase?
9. Why does `printDesc(3)` print `3 2 1`?
10. Why does `printAsc(3)` print `1 2 3`?
11. What is the output of a function that prints before and after the recursive call?
12. Why should you trace recursive code before running it?

---

18. Final Rule for Week 1

Do not memorize recursion code blindly.

Always ask four questions:

1. What is the base case?
2. What is the recursive case?
3. What happens before the recursive call?

4. What happens after the recursive call?

If you can answer these four questions, you can understand most beginner recursion examples.